

BYOP: AI Language Detection System

Final Project Documentation

Author: Srinivas.S.S
Registration No: 25BCE10481
Project Category: Natural Language Processing (NLP)

1. Project Abstract

The "BYOP: AI Language Detection System" is a specialized software application designed to categorize and identify the linguistic origin of digital text. In an era of global connectivity, this tool serves as a bridge, allowing users to instantly recognize text from over a dozen major world languages. By leveraging statistical probability models, the system provides a nuanced analysis rather than a simple guess, making it a valuable educational resource for understanding computational linguistics.

2. Technical Motivation

The primary motivation behind this project was to explore the intersection of human language and computer science. Traditional coding often deals with rigid structures, but human language is fluid and overlapping. I wanted to build a system that could:

- Handle "ambiguous" words that exist in multiple languages.
- Provide a transparent confidence rating for every analysis performed.
- Offer a professional Command Line Interface (CLI) for rapid data testing.

3. Core Functionalities

The system is built with several robust features to ensure a high-quality user experience:

- **Heuristic Pattern Scanning:** The AI scans for specific character clusters (N-grams) that are unique to certain language families.
- **Batch Processing:** The application includes a file-handling module that allows users to upload .txt documents for full-page language auditing.
- **Probability Distribution:** To ensure transparency, the system displays the "Top 3" most likely languages. This is crucial when the AI identifies a 55% chance of German but a 40% chance of Dutch.

- **Local Persistence:** Through a `history.json` integration, the app maintains a record of recent activities, allowing for a longitudinal review of previous detections.

4. Methodology & Logic

The application utilizes the *langdetect* library, which is a Python implementation of the renowned language-detection library used in various search engine technologies.

1. **Input Normalization:** The text is cleaned and stripped of unnecessary symbols to focus on linguistic characters.
2. **Profile Comparison:** The input is compared against pre-defined language profiles created from large datasets of global literature.
3. **Deterministic Execution:** By implementing a fixed seed (`DetectorFactory.seed = 0`), the project ensures mathematical consistency, meaning identical inputs will always produce identical confidence scores.

5. Personal Learning & Reflection

Building this project was a significant learning curve in terms of error handling and user interface design. One of the most interesting challenges was managing "Short Text Uncertainty." I learned that AI requires a certain "critical mass" of data to be accurate. If a user provides too little information, the system is designed to trigger a "Smart Warning," advising the user to provide more context. This taught me that good software doesn't just give answers—it guides the user toward better results.

6. Future Scope

In future iterations, I hope to expand the system to include non-Latin scripts (such as Devanagari or Cyrillic) more extensively and perhaps integrate a translation API. This would turn the detector into a full-scale communication suite.

7. Conclusion

This project successfully demonstrates the power of Python in solving real-world linguistic problems. It bridges the gap between raw machine learning data and a usable, formal interface. As my BYOP submission, this detector represents a functional application of Natural Language Processing principles.